Knowledge Representation of a Production Release Approval Process

AIMLZG557 Artificial and Computational Intelligence, S1-26 · Assignment 2, Question 2 (Situated Learning, individual) · Dr. V Indumathi

B.1 Submission Form

Field	Entry
Name	Shekhar Aggarwal
ID No	2025AG05925
Industry / Domain	Software development
Organisation Type	Private IT services company
[Instructor prompt]	Not provided

B.2 Anchor: the workplace process

IT services companies build and run software for external clients, under contracts that set quality, security and uptime commitments. Every change that goes to a client's production system is therefore controlled by a **production release approval** process, also called change management. A release is one tested set of changes, such as a sprint's features or a hotfix, that is to be deployed to a client's live environment.

The process decides one question for every release: **can it go to production now?** The decision depends on fixed, written rules: who reviewed the code, whether the automated tests and QA passed, whether the database is touched, whether the client accepted the changes in UAT (user acceptance testing), whether the Change Advisory Board (CAB) approved it, and whether a release freeze is in force. A separate fast path exists for emergency fixes to Severity-1 incidents, which cannot wait for the normal approvals.

This process suits a rule-based knowledge base well. The decisions are already governed by rules rather than judgement, the facts come from tools the team already uses (code review, CI pipeline, ticketing), and every decision must be explainable afterwards, for example in a client audit or a post-incident review.

The rules below are written as they commonly apply in IT services companies. Names of people, releases, incidents and clients are anonymised: D1 to D3 are developers, DM1 is a delivery manager, R1 to R5 are releases, INC7 is an incident and ClientA and ClientB are clients.

B.3 Apply

1. Business rules

Rule	Business rule
BR1	**Peer review.** A release is peer-reviewed only if someone other than its author has reviewed the code.
BR2	**Quality gate.** A release is quality-cleared when it is peer-reviewed, all automated tests pass and QA has signed it off.
BR3	**No schema change.** A release that does not change the database schema is data-safe.
BR4	**Schema change.** A release that changes the database schema is data-safe only after a DBA has reviewed it.
BR5	**Security.** A release with an open critical security vulnerability is blocked.

Rule	Business rule
BR6	**Normal approval.** A normal release is approved when it is quality-cleared, data-safe, has a rollback plan, the client has signed off UAT and the Change Advisory Board (CAB) has approved it.
BR7	**Valid emergency.** A release marked as an emergency fix is a valid emergency only if it fixes a Severity-1 incident.
BR8	**Emergency approval.** A valid emergency fix skips QA sign-off, client UAT and the CAB. It is approved when it is peer-reviewed, its tests pass, it has a rollback plan and a delivery manager has approved it.
BR9	**Release freeze.** During a release freeze, any release that is not a valid emergency is blocked.
BR10	**Deployment.** A release can be deployed to production when it is approved and not blocked.
BR11	**Post-incident review.** An emergency fix that is deployed requires a post-incident review.
BR12	**Client notice.** The client of every release that will be deployed must be notified.

2. Propositional logic

In propositional logic each statement about *the release under consideration* becomes one proposition:

Symbol	Meaning
RN	The release was reviewed by someone other than its author
PR	The release is peer-reviewed
T	All automated tests pass
QA	QA has signed off the release
QC	The release is quality-cleared
SC	The release changes the database schema
DBA	A DBA has reviewed the release
DS	The release is data-safe
CV	The release has an open critical security vulnerability
BL	The release is blocked
RB	The release has a documented rollback plan
UAT	The client has signed off UAT
CAB	The CAB has approved the release
AP	The release is approved
EM	The release is marked as an emergency fix
S1	The release fixes a Severity-1 incident
VE	The release is a valid emergency
DM	A delivery manager has approved the release
FW	The release falls in a freeze window
CD	The release can be deployed to production
PIR	A post-incident review is required
NC	The client must be notified

Rule	Propositional form
BR1	`RN → PR`
BR2	`PR ∧ T ∧ QA → QC`

Rule	Propositional form
BR3	¬SC → DS
BR4	SC ∧ DBA → DS
BR5	CV → BL
BR6	QC ∧ DS ∧ RB ∧ UAT ∧ CAB → AP
BR7	EM ∧ S1 → VE
BR8	VE ∧ PR ∧ T ∧ RB ∧ DM → AP
BR9	FW ∧ ¬VE → BL
BR10	AP ∧ ¬BL → CD
BR11	CD ∧ VE → PIR
BR12	CD → NC

Each proposition describes a single release, so checking five releases needs a separate copy of every symbol for each one, such as T_{R1} and T_{R2}. The proposition RN also has to be decided outside the logic, because propositional logic cannot express "someone other than the author".

3. Predicate logic

Constants: releases R1 to R5, people D1, D2, D3 and DM1, incident INC7, clients ClientA and ClientB. **Variables:** r (release), a and p (people), m (manager), i (incident), c (client). **Quantifiers:** ∀ makes each rule hold for every release. ∃ is used where a rule needs *at least one* object to exist: a reviewer other than the author (BR1), a Severity-1 incident (BR7) and an approving delivery manager (BR8). BR1 also uses the inequality p ≠ a.

Predicate	Arity	Meaning
Release(r)	1	r is a release
Author(r, a)	2	a wrote the code of release r
ReviewedBy(r, p)	2	p reviewed the code of r
TestsPassed(r)	1	all automated tests of r pass
QASignedOff(r)	1	QA signed off r
SchemaChange(r)	1	r changes the database schema
DBAReviewed(r)	1	a DBA reviewed r
CriticalVulnerability(r)	1	r has an open critical vulnerability
RollbackPlan(r)	1	r has a rollback plan
ClientUATApproved(r)	1	the client signed off UAT for r
CABApproved(r)	1	the CAB approved r
Emergency(r)	1	r is marked as an emergency fix
FixesIncident(r, i)	2	r fixes incident i
Severity1(i)	1	incident i is Severity 1

Predicate	Arity	Meaning
ApprovedBy(r, m)	2	m approved r
DeliveryManager(m)	1	m is a delivery manager
InFreezeWindow(r)	1	r is planned inside a freeze window
ForClient(r, c)	2	r is delivered to client c
PeerReviewed, QualityCleared, DataSafe, Blocked, Approved, ValidEmergency, CanDeploy, RequiresPostIncidentReview (r)	1	derived by the rules
MustNotify(c, r)	2	derived: client c must be told about r

Rule	Predicate-logic form
BR1	`∀r ∀a (Author(r, a) ∧ ∃p (ReviewedBy(r, p) ∧ p ≠ a) → PeerReviewed(r))`
BR2	`∀r (PeerReviewed(r) ∧ TestsPassed(r) ∧ QASignedOff(r) → QualityCleared(r))`
BR3	`∀r (Release(r) ∧ ¬SchemaChange(r) → DataSafe(r))`
BR4	`∀r (SchemaChange(r) ∧ DBAReviewed(r) → DataSafe(r))`
BR5	`∀r (CriticalVulnerability(r) → Blocked(r))`
BR6	`∀r (QualityCleared(r) ∧ DataSafe(r) ∧ RollbackPlan(r) ∧ ClientUATApproved(r) ∧ CABApproved(r) → Approved(r))`
BR7	`∀r (Emergency(r) ∧ ∃i (FixesIncident(r, i) ∧ Severity1(i)) → ValidEmergency(r))`
BR8	`∀r (ValidEmergency(r) ∧ PeerReviewed(r) ∧ TestsPassed(r) ∧ RollbackPlan(r) ∧ ∃m (ApprovedBy(r, m) ∧ DeliveryManager(m)) → Approved(r))`
BR9	`∀r (InFreezeWindow(r) ∧ ¬ValidEmergency(r) → Blocked(r))`
BR10	`∀r (Approved(r) ∧ ¬Blocked(r) → CanDeploy(r))`
BR11	`∀r (CanDeploy(r) ∧ ValidEmergency(r) → RequiresPostIncidentReview(r))`
BR12	`∀r ∀c (CanDeploy(r) ∧ ForClient(r, c) → MustNotify(c, r))`

4. Knowledge base: facts

The knowledge base holds 50 ground facts about five releases that are waiting for deployment in the same week.

#	Fact	#	Fact
F1	`Release(R1)`	F26	`RollbackPlan(R3)`
F2	`Release(R2)`	F27	`Emergency(R3)`
F3	`Release(R3)`	F28	`FixesIncident(R3, INC7)`
F4	`Release(R4)`	F29	`Severity1(INC7)`
F5	`Release(R5)`	F30	`ApprovedBy(R3, DM1)`
F6	`Author(R1, D1)`	F31	`DeliveryManager(DM1)`
F7	`ReviewedBy(R1, D2)`	F32	`InFreezeWindow(R3)`
F8	`TestsPassed(R1)`	F33	`ForClient(R3, ClientA)`
F9	`QASignedOff(R1)`	F34	`Author(R4, D1)`
F10	`RollbackPlan(R1)`	F35	`ReviewedBy(R4, D3)`

#	Fact	#	Fact
F11	`ClientUATApproved(R1)`	F36	`TestsPassed(R4)`
F12	`CABApproved(R1)`	F37	`QASignedOff(R4)`
F13	`ForClient(R1, ClientA)`	F38	`SchemaChange(R4)`
F14	`Author(R2, D2)`	F39	`RollbackPlan(R4)`
F15	`ReviewedBy(R2, D3)`	F40	`ClientUATApproved(R4)`
F16	`TestsPassed(R2)`	F41	`CABApproved(R4)`
F17	`QASignedOff(R2)`	F42	`ForClient(R4, ClientB)`
F18	`RollbackPlan(R2)`	F43	`Author(R5, D2)`
F19	`ClientUATApproved(R2)`	F44	`ReviewedBy(R5, D2)`
F20	`CABApproved(R2)`	F45	`TestsPassed(R5)`
F21	`CriticalVulnerability(R2)`	F46	`QASignedOff(R5)`
F22	`ForClient(R2, ClientB)`	F47	`RollbackPlan(R5)`
F23	`Author(R3, D3)`	F48	`ClientUATApproved(R5)`
F24	`ReviewedBy(R3, D1)`	F49	`CABApproved(R5)`
F25	`TestsPassed(R3)`	F50	`ForClient(R5, ClientB)`

5. Knowledge base: inference rules

The inference rules are the predicate rules rewritten as definite clauses (Horn form), which is the form a forward- or backward-chaining engine uses. An existential quantifier in the condition becomes a universal one over the whole rule, because $(\exists x\ P(x)) \rightarrow Q$ is equivalent to $\forall x\ (P(x) \rightarrow Q)$. The condition *not X* is read under the **closed-world assumption**: X is taken to be false when it cannot be derived from the knowledge base. The rules are stratified, so each *not* refers only to facts or to conclusions that are complete before the rule is used.

#	Inference rule	From
IR1	`∀r ∀a ∀p (Author(r, a) ∧ ReviewedBy(r, p) ∧ p ≠ a → PeerReviewed(r))`	BR1
IR2	`∀r (PeerReviewed(r) ∧ TestsPassed(r) ∧ QASignedOff(r) → QualityCleared(r))`	BR2
IR3	`∀r (Release(r) ∧ not SchemaChange(r) → DataSafe(r))`	BR3
IR4	`∀r (SchemaChange(r) ∧ DBAReviewed(r) → DataSafe(r))`	BR4
IR5	`∀r (CriticalVulnerability(r) → Blocked(r))`	BR5
IR6	`∀r (QualityCleared(r) ∧ DataSafe(r) ∧ RollbackPlan(r) ∧ ClientUATApproved(r) ∧ CABApproved(r) → Approved(r))`	BR6
IR7	`∀r ∀i (Emergency(r) ∧ FixesIncident(r, i) ∧ Severity1(i) → ValidEmergency(r))`	BR7
IR8	`∀r ∀m (ValidEmergency(r) ∧ PeerReviewed(r) ∧ TestsPassed(r) ∧ RollbackPlan(r) ∧ ApprovedBy(r, m) ∧ DeliveryManager(m) → Approved(r))`	BR8
IR9	`∀r (InFreezeWindow(r) ∧ not ValidEmergency(r) → Blocked(r))`	BR9
IR10	`∀r (Approved(r) ∧ not Blocked(r) → CanDeploy(r))`	BR10
IR11	`∀r (CanDeploy(r) ∧ ValidEmergency(r) → RequiresPostIncidentReview(r))`	BR11
IR12	`∀r ∀c (CanDeploy(r) ∧ ForClient(r, c) → MustNotify(c, r))`	BR12

Inference uses universal instantiation (replacing variables with constants from the facts) followed by modus ponens.

6. Answering workplace queries

Q1. Can release R1 be deployed to production?

Goal: `CanDeploy(R1)`

1. IR1 with r = R1, a = D1, p = D2: `Author(R1, D1)` (F6), `ReviewedBy(R1, D2)` (F7) and D2 ≠ D1, so `PeerReviewed(R1)`.
2. IR2: `PeerReviewed(R1)`, `TestsPassed(R1)` (F8) and `QASignedOff(R1)` (F9) give `QualityCleared(R1)`.
3. IR3: `Release(R1)` (F1) and there is no `SchemaChange(R1)` fact, so `DataSafe(R1)`.
4. IR6: `QualityCleared(R1)`, `DataSafe(R1)`, F10, F11 and F12 give `Approved(R1)`.
5. IR5 and IR9 cannot fire for R1: there is no critical vulnerability and no freeze window. So `Blocked(R1)` cannot be derived.
6. IR10: `Approved(R1)` and not `Blocked(R1)` give `CanDeploy(R1)`.

Answer: **Yes.** R1 passed every gate on the normal path.

Q2. Can release R2 be deployed to production?

Goal: `CanDeploy(R2)`

1. As in Q1, IR1, IR2, IR3 and IR6 give `Approved(R2)` (from F14, F15, F16, F17, F2, F18, F19 and F20).
2. IR5: `CriticalVulnerability(R2)` (F21) gives `Blocked(R2)`.
3. IR10 needs not `Blocked(R2)`, which is false, so `CanDeploy(R2)` cannot be derived.

Answer: **No.** R2 has every approval, but the critical vulnerability blocks it. Approval alone is not enough.

Q3. Can the emergency fix R3 be deployed during the freeze, and what follows?

Goal: `CanDeploy(R3), RequiresPostIncidentReview(R3)`

1. IR7 with r = R3, i = INC7: `Emergency(R3)` (F27), `FixesIncident(R3, INC7)` (F28) and `Severity1(INC7)` (F29) give `ValidEmergency(R3)`.
2. IR1: F23 and F24 with D1 ≠ D3 give `PeerReviewed(R3)`.
3. IR8 with m = DM1: `ValidEmergency(R3)`, `PeerReviewed(R3)`, F25, F26, `ApprovedBy(R3, DM1)` (F30) and `DeliveryManager(DM1)` (F31) give `Approved(R3)`. QA sign-off, UAT and CAB are not needed on this path.
4. IR9: R3 is in a freeze window (F32), but `ValidEmergency(R3)` holds, so the rule does not fire. Nothing else blocks R3.
5. IR10 gives `CanDeploy(R3)`, and IR11 then gives `RequiresPostIncidentReview(R3)`.

Answer: **Yes.** R3 may go out during the freeze because it fixes a Severity-1 incident, and a post-incident review is now required.

Q4. Is release R5 peer-reviewed, and can it be deployed?

Goal: `PeerReviewed(R5), CanDeploy(R5)`

1. The only review fact for R5 is `ReviewedBy(R5, D2)` (F44), and the author is also D2 (F43).
2. IR1 needs a reviewer p with p ≠ a. The only candidate is p = D2 = a, so IR1 cannot fire and PeerReviewed(R5) cannot be derived.
3. Without `PeerReviewed(R5)`, IR2 cannot give `QualityCleared(R5)`, so IR6 cannot give `Approved(R5)`. R5 is not an emergency, so IR8 does not apply either.
4. IR10 needs `Approved(R5)`, so `CanDeploy(R5)` cannot be derived.

Answer: **No to both.** R5 has every other approval, but its author reviewed it alone, which does not count as a peer review.

Q5. Which clients must be notified of a deployment?

Goal: `∃r MustNotify(c, r), solved for c`

1. IR12 fires only for releases with CanDeploy(r). From Q1 to Q4, CanDeploy holds for R1 and R3, and not for R2 or R5.
2. R4 changes the schema (F38) and there is no DBAReviewed(R4) fact. So neither IR3 nor IR4 gives DataSafe(R4), IR6 cannot give Approved(R4), and CanDeploy(R4) cannot be derived.
3. IR12 with r = R1: F13 gives `MustNotify(ClientA, R1)`.
4. IR12 with r = R3: F33 gives `MustNotify(ClientA, R3)`.

Answer: **ClientA**, about R1 and R3. ClientB is not notified, because none of its releases (R2, R4, R5) can be deployed.

B.4 Analyse

How predicate logic improves on propositional logic here

- **One rule covers every release.** In propositional logic, the 12 rules have to be copied for each release, which is 60 rules for five releases, with new symbols for each one. In predicate logic the same 12 rules hold for any number of releases. A new release only adds facts.
- **It can express relations between objects.** BR1 depends on *who* reviewed the code compared with *who* wrote it. Propositional logic can only take RN as given. Predicate logic checks it with Author(r, a), ReviewedBy(r, p) and p ≠ a, which is how it catches the self-review in Q4.
- **It can say "at least one".** BR7 and BR8 need some Severity-1 incident and some delivery manager to exist. Predicate logic expresses this directly with ∃ and finds the actual incident and manager while reasoning (INC7 and DM1 in Q3).
- **It answers "which" questions.** Propositional logic can only answer yes or no about NC. Predicate logic finds the actual client in Q5, ClientA, and links each notice to the release it is about.

How well the knowledge base meets the workplace need

What the release process needs	Result	How
A clear go or no-go for each release	Met	CanDeploy is derived or not for every release (Q1 to Q4)
The reason for a no-go	Met	The derivation shows which rule failed, such as the vulnerability for R2 or the self-review for R5
Who must be told	Met	MustNotify names the client and the release (Q5)

What the release process needs	Result	How
Fast handling of production emergencies	Met	IR7 and IR8 give a separate, shorter approval path that still keeps review and rollback
Follow-up after emergencies	Partly met	A review is flagged as required, but nothing tracks a deadline or whether it happened
Risk-based decisions (partial test failures, medium vulnerabilities)	Not met	Every condition is true or false, so there is no idea of degree or risk

Missing knowledge and limitations

- **The closed-world assumption can be unsafe.** It works well for approvals: a missing approval means "not approved", which fails safe. It fails unsafe for risks. If a schema change is simply not recorded, IR3 treats the release as data-safe. Safety facts like SchemaChange and CriticalVulnerability must be captured reliably, not entered by hand.
- **No time.** Freeze windows, approval expiry (a CAB approval from last month should not count) and review deadlines all depend on dates, which these rules cannot represent.
- **No numbers or degrees.** Real gates use thresholds, such as at least 80% test coverage or no high-severity vulnerabilities, and a risk level decides how many approvals are needed. Here every condition is simply true or false.
- **No exceptions or client-specific policy.** Clients often have their own rules, such as extra sign-offs or blocked deployment days. Adding them would need rule priorities or separate rule sets per client.
- **No roles or delegation.** The knowledge base does not check that DM1 manages this particular project, or allow a deputy to approve while a manager is away.

B.5 Reflect

Practical constraints on deployment

- **Incomplete knowledge.** The reasoning is only as good as its facts, and the facts are spread across the code review tool, the CI pipeline, the ticketing system and email approvals. Any fact that is missing or out of date changes the answer, and under the closed-world assumption it does so without warning.
- **Changing business rules.** Freeze calendars, client contracts and security policies change, and each client may differ. Rules need an owner, version control and testing, just like code.
- **Privacy and confidentiality.** The knowledge base holds client names, incident details and who approved what. In a services company, one client's information must not be visible to people working for another client, so access has to be restricted per client.
- **Scale.** Reasoning over a few hundred releases is cheap. The real cost is keeping hundreds of client-specific rules consistent without contradictions, such as one rule blocking a release that another requires.
- **Technology and trust.** The engine has to fit into existing release tools, and release managers will rely on it only if every decision comes with the reasoning behind it.

The most important improvement before deployment

Feed the facts automatically from the systems of record, and treat a missing safety fact as unknown, which blocks the release, rather than as false. For example, SchemaChange would be detected from the database migration files in the release, TestsPassed and CriticalVulnerability would come from the CI pipeline, and approvals would come from the ticketing system.

This matters most because the rules themselves are simple and easy to check, while the facts are where the knowledge base can go silently wrong. A single missing SchemaChange fact makes IR3 approve a risky database change, with nothing in the output to show the mistake. Automatic facts remove manual entry

errors, and failing closed on missing safety facts means a gap in knowledge leads to a held release, not a production incident.

What I learned

Writing the process as logic showed how much of it is left unstated in practice. "Peer review" turned out to need an explicit rule that the reviewer is not the author, and "approved" and "can be deployed" turned out to be different conclusions, as release R2 shows. Moving from propositional to predicate logic made the rules shorter and more general, and it let the knowledge base answer "which client" rather than only "yes or no". The biggest lesson was that the closed-world assumption is a design decision with real safety consequences, not just a technical detail.